JAVA INTERVIEW PREPARATION

CHAPTER 62

HTTP 500 ERRORS AND
EXCEPTION DIAGNOSIS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Diagnosing HTTP 500 Errors and Exception Storms

Senior / Lead Java Developer Reading Chapter

An HTTP 500 response means the server failed to complete a request as promised, but it does not identify the failing component. The response may be created by application exception handling, the servlet container, a reverse proxy, a gateway, a service mesh, or a platform function. A useful investigation identifies who generated the status, what operation failed, and whether the failure is isolated or systemic.

The production goal is not merely to remove stack traces. It is to restore correct service, preserve evidence, prevent retry amplification, and correct the ownership or invariant that allowed an internal failure to escape.

```
client -> edge/gateway -> Java service -> database/downstream
          502/503/504?      500?          timeout/error?

First locate the layer that authored the response.
```

Begin with precise scope

Define the incident by:

- normalized endpoint and HTTP method;
- status code and response signature;
- start time and rate, not only total count;
- affected version, instance, zone, region, and tenant class;
- request volume, payload size, and authentication path;
- correlation with deployment, configuration, schema, traffic, or dependency changes.

A rise from one to ten errors per minute has different meaning at ten requests per minute than at one million. Track error ratio, absolute rate, and number of affected users or business operations.

Separate 500 from nearby statuses. A gateway may return 502 for an invalid upstream response, 503 when no healthy capacity is available, or 504 when an upstream deadline expires. Exact behavior depends on the gateway, but treating every 5xx as an application exception wastes investigation time.

Protect service and preserve evidence

If a recent rollout correlates strongly, pause or revert it while keeping one failing artifact and trace sample. Other mitigations include disabling the affected feature, routing away from a bad zone, limiting a toxic payload pattern, pausing bulk work, or degrading an optional dependency.

Before restarting all instances, preserve:

```
request/correlation ID and UTC time
route template, status, version, instance, region
sanitized error response signature
exception type and causal chain
representative trace and bounded log context
CPU, memory, threads, pools, GC, dependency health
deployment/config/schema changes
```

Do not copy raw customer payloads into an incident channel. Logs and traces may contain credentials, personal data, and financial details. Use approved secure retrieval and redact by design.

Determine who produced the response

Compare client, edge, gateway, and service telemetry. A response header or standardized problem body may identify the author, but it should not be trusted blindly if intermediaries rewrite responses.

If the Java service has no matching request record, investigate routing, gateway policy, TLS, connection reset, request-size enforcement, or logging loss. If the service records a 200 but the client receives 500, inspect response transfer, proxy behavior, and whether the connection failed after headers.

If the service records a 500, locate the exception translation path. In Spring MVC this may involve an `@ExceptionHandler`, `@ControllerAdvice`, `HandlerExceptionResolver`, container error dispatch, or Spring Boot's error handling. The final status can differ from the originally thrown exception.

Use a stable error contract

Clients should receive a safe, predictable error shape with a correlation identifier and a stable machine-readable problem type. Do not expose class names, SQL, internal URLs, stack traces, or secrets.

Spring supports RFC 9457-style `ProblemDetail` and `ErrorResponse` abstractions. A centralized handler can translate domain and infrastructure failures deliberately:

```
@RestControllerAdvice
final class ApiErrors {

    @ExceptionHandler(OrderNotFoundException.class)
    ResponseEntity<ProblemDetail> notFound(
            OrderNotFoundException ex, HttpServletRequest request) {
        ProblemDetail p = ProblemDetail.forStatus(404);
        p.setType(URI.create("https://errors.example.com/order-not-found"));
        p.setTitle("Order not found");
        p.setDetail("The requested order does not exist or is unavailable.");
        p.setProperty("correlationId", currentCorrelationId());
        return ResponseEntity.status(404).body(p);
    }

    @ExceptionHandler(Exception.class)
    ProblemDetail unexpected(Exception ex) {
        recordUnexpected(ex); // controlled structured event, not raw payload
        ProblemDetail p = ProblemDetail.forStatus(500);
        p.setTitle("Internal server error");
        p.setProperty("correlationId", currentCorrelationId());
        return p;
    }
}
```

The broad fallback is a containment boundary, not a substitute for specific classification. Handler ordering and framework behavior must be tested, including exceptions thrown before controller invocation and after the response is committed.

Classify the failure before fixing it

A practical taxonomy is:

- deterministic input or state: the same request always fails;
- deployment or compatibility: only a new version or schema combination fails;
- dependency: database, cache, broker, identity, or remote API fails;
- resource exhaustion: threads, connections, heap, disk, file descriptors, or CPU;
- concurrency or race: intermittent timing-dependent failure;
- data-specific: one tenant, record, encoding, size, or historical version;

- operational configuration: secret, certificate, feature flag, route, or permission.

The exception class alone does not supply this classification. A `NullPointerException` may be deterministic corrupt data, a deployment mismatch, or a race. A timeout may be downstream slowness or local connection-pool starvation.

Read the full causal chain

Start with the top-level exception that reached the boundary, but follow `cause` and relevant suppressed exceptions. Framework wrappers such as transaction, reflection, completion, or servlet exceptions often hide the actionable cause.

```
CompletionException
  -> PaymentClientException
     -> SocketTimeoutException
```

Avoid logging the same chain at every layer. One failure logged by client, service, controller advice, and gateway can create four large stack traces and an I/O storm. Intermediate layers should add structured context or translate, while the owning boundary records the diagnostic event once.

Preserve exception identity when translating. Throwing `new RuntimeException("failed")` without the cause destroys the stack and causal chain.

Correlation IDs, traces, and logs

A correlation ID joins client reports, edge access logs, service logs, traces, and downstream calls. Accept an incoming ID only through a trusted boundary or validate its format; otherwise clients can inject misleading or high-cardinality values.

Structured error events should include bounded fields:

```
timestamp, service, version, instance, region
route template, method, status, exception category
dependency name, attempt number, elapsed time
correlation/trace ID, feature version
```

Do not use raw URL, user ID, stack trace, or exception message as metric labels. Metrics aggregate; logs and sampled traces hold detailed investigation context.

Trace the full logical operation, including retries. One client request may create several server attempts. A final 500 trace should show which attempt failed and how much deadline remained.

Error fingerprints and incident grouping

One exception type can have many causes, while several wrapper types can share one cause. Group errors using a bounded fingerprint derived from stable attributes such as owning service, normalized route, exception chain classes, and selected application frame. Do not include line numbers alone because builds change them, and do not include messages containing IDs.

```
fingerprint inputs
  = route template
  + top application exception categories
  + first stable owning frame
  + dependency category where applicable
```

Track count, first seen, last seen, affected versions, and representative trace IDs for each fingerprint. This distinguishes one widespread defect from thousands of unique customer data failures.

Fingerprinting is an operational aid, not proof of identical root cause. Revisit grouping when one bucket spans different payload or dependency patterns. Preserve a small number of secure examples from each significant subgroup.

Security and authorization failures

Authentication and authorization failures should normally become 401 or 403 according to the contract, not 500. A broken identity provider, key refresh, policy engine, or tenant lookup can nevertheless cause server failures.

Fail closed for protected operations, but distinguish a denied identity from unavailable authorization infrastructure in internal telemetry. Do not leak whether another tenant's resource exists. Security exception messages and tokens must never be copied into problem details.

An exception handler must not bypass security filters or convert access denial into success. Test filter-chain failures, expired and malformed tokens, missing keys, method security, tenant mismatch, and identity-provider timeout. Audit security-relevant failure decisions through a protected channel with bounded fields.

Error endpoints themselves need protection. A client should not force expensive stack formatting, reflection, localization, or database lookups simply by generating invalid requests.

Metrics can hide handled exceptions

Spring Boot instruments server requests through observations and commonly publishes `http.server.requests`. Exception information may be absent when an exception is handled inside a controller or resolver unless the application records it through the supported mechanism.

Do not assume a metric tagged with `exception=none` proves no exception occurred. Compare status, dedicated error counters, handler telemetry, and traces. Test the observability path by intentionally triggering each error category in a nonproduction environment.

Measure:

- request count and 5xx ratio by route, status, version, and region;
- unexpected exception rate by bounded category;
- dependency timeout and error rate;
- retries per original request;
- error-handler failures and response-serialization errors;
- log volume and dropped-log count;
- business failures and reconciliation mismatches.

Deployment and compatibility failures

A sharp error increase after rollout strongly suggests but does not prove regression. Compare old and new versions under the same traffic. Inspect configuration, environment, feature flags, library versions, and schema compatibility.

Rolling deployments create mixed versions. A new producer may publish an event or database value an old consumer cannot read. A destructive migration may remove a column still used by old instances. An API client and server can disagree on enum values or required fields.

Use expand-and-contract changes and backward-compatible contracts. Canary by real traffic and monitor route-specific errors, not only process health. Readiness can pass while a rarely used path is broken.

Data-specific and payload-specific failures

If the error affects one tenant or record, compare sanitized structural properties: schema version, field presence, character encoding, size, cardinality, and lifecycle state. Do not assume the data is "bad" merely because code cannot handle it; historical valid data may expose an untested compatibility assumption.

Reproduce with a minimized, anonymized fixture. Preserve the original securely for forensic verification. Add validation at the trust boundary and maintain a quarantine or repair path when asynchronous data cannot be processed.

Payload limits should be enforced before expensive deserialization, decompression, or allocation. Compression bombs and deeply nested structures can cause resource failures that appear as ordinary 500s unless translated safely.

Database failures

Database-related 500s may originate from:

- connection acquisition timeout;
- statement timeout or lock wait;
- constraint violation;
- deadlock victim selection;
- serialization failure;
- schema mismatch;
- stale connection or failover;
- ORM lazy access outside a persistence context.

Classify constraint violations. A uniqueness conflict during idempotent creation may be a successful replay or a 409 conflict, not an internal error. A database unavailable condition may deserve 503 and a retry policy; a malformed query is an application defect.

Record the statement fingerprint and database error code through secure tooling, not full values. Inspect pool acquisition separately from execution. Retrying a nontransient constraint or syntax failure amplifies the incident.

Downstream and timeout failures

A remote timeout can be translated to 500 accidentally, even though 502, 503, or 504 may better describe the boundary. The precise external status is an API contract decision; internal classification must still preserve dependency, timeout phase, attempt, and remaining deadline.

Retries require idempotency, transient classification, jitter, and a total budget. Nested retries can create exception storms and make a small dependency outage look like a Java CPU or logging outage.

Fallbacks must be semantically safe. Returning an empty permissions list, zero account balance, or "success" after a failed write can be worse than a clear error. Degrade only optional data with an explicit response contract.

Resource exhaustion masquerading as exceptions

Thread exhaustion, connection-pool starvation, heap pressure, disk full, file-descriptor limits, and CPU saturation often surface through many unrelated exceptions. If several routes and dependencies fail together, look for a shared local resource.

Examples include:

- `RejectedExecutionException` from a saturated executor;
- connection acquisition timeout across unrelated queries;
- logging failure because disk is full;
- `OutOfMemoryError` followed by secondary handler failures;
- inability to create threads or sockets;
- timeouts because container CPU is throttled.

Do not debug each stack independently when the common resource explains the time correlation.

Response serialization and committed responses

The controller may return successfully and then fail while serializing the body. Causes include lazy ORM proxies, recursive object graphs, custom serializer defects, invalid encodings, or a client disconnect.

If response headers are already committed, the server may be unable to replace the partial body with a clean problem response. The client can observe a truncated body or connection reset rather than a normal 500.

Keep API DTOs separate from persistence entities, bound graph depth and size, and test serialization. Streaming endpoints need separate error semantics because failure after partial output cannot be represented as an ordinary replacement response.

Asynchronous and reactive exception paths

Exceptions in `CompletableFuture`, executor tasks, callbacks, Reactor chains, or virtual threads may not reach the original controller boundary automatically. A missing terminal error handler can log late, disappear into a future, or cause a timeout instead of a direct 500.

Preserve observation context and ownership across asynchronous boundaries. Define who records the failure, who completes the response, and who cancels underlying work. Avoid blocking reactive event-loop threads with synchronous error logging or data access.

An HTTP timeout does not prove the background operation stopped. Reconciliation must handle ambiguous writes that completed after the client received an error.

Ambiguous writes and client retry behavior

A 500 response does not reliably mean "nothing happened." The database may commit and response serialization may fail; a remote provider may accept the command and the network may drop the acknowledgement; a transaction may time out from the caller's perspective while the final outcome is not yet known.

For retriable commands, use a client-supplied or server-issued idempotency key persisted with the business result. On retry, return the existing result or a stable in-progress status rather than executing again. A database uniqueness constraint should enforce the identity across service instances.

```
operation ID -> IN_PROGRESS -> SUCCEEDED(result reference)
                         \-> FAILED_RETRYABLE / FAILED_FINAL
```

The state model must handle a crash between business side effect and status update. Prefer atomic local transactions or an outbox where possible; otherwise reconcile with the external system using the provider reference.

Clients should retry only operations declared safe and only transient categories, with jitter and a bounded total deadline. A generic SDK policy that retries every 500 can duplicate writes and amplify deterministic defects.

Error budgets and release decisions

An error budget connects 5xx rates to the availability objective. It does not excuse individual correctness defects, but it informs rollout speed and operational priority. A deployment that rapidly consumes the budget should automatically pause or roll back when the signal is trustworthy.

Use route and business criticality. Ten errors on payment authorization can matter more than a thousand failures in optional recommendations. Track successful business completion, not only HTTP status; a handler returning 200 with an error field falsely improves the technical SLI.

Burn-rate alerts over short and long windows distinguish fast incidents from gradual degradation. The alert should link to version, route, dependency, and representative error fingerprints so responders begin with evidence rather than a generic dashboard.

Exception storms and logging collapse

When thousands of requests fail, generating and formatting stack traces, capturing large contexts, and synchronously writing logs can consume CPU, heap, disk, and network. The error path becomes a second outage.

Use structured bounded events, aggregation, sampling or rate limits for repeated identical failures, and a counter that preserves total magnitude. Always retain enough unsampled examples and causal context for diagnosis.

Do not suppress all errors globally. Separate expected client/domain rejections from unexpected server failures, and alert on changes in category and business impact.

A disciplined investigation sequence

```
1. confirm exact status and response author
2. scope by route, version, instance, region, tenant, payload
3. preserve representative trace, causal chain, and resource state
4. classify deterministic, dependency, resource, race, data, or compatibility
5. correlate deployment/config/schema and shared-resource signals
6. mitigate retries and customer impact
7. reproduce with sanitized fixture or failure injection
8. correct ownership/translation and verify observability
9. reconcile ambiguous business outcomes
```

This sequence prevents a common mistake: fixing the first visible stack trace while the shared dependency or resource remains broken.

Verify the correction

Test success and error paths. Assert status, problem type, safe detail, correlation ID, log count, metric classification, trace error status, and absence of sensitive fields. Exercise validation, dependency timeout, database conflict, serialization failure, asynchronous exception, and error-handler failure.

Run a canary with the same payload and traffic dimensions. Verify that 500 rate falls without incorrectly converting failures into 200 responses. Check business reconciliation for operations that may have committed before an error reached the client.

Add a regression test and a bounded alert on the owning category. A generic 5xx alert is necessary but insufficient for rapid diagnosis.

Common senior-level traps

"The stack trace says where the root cause is." Wrappers, shared resources, and downstream failures require causal and timeline analysis.

"Catch `Exception` and return 200 with an error field." This corrupts HTTP semantics, observability, retry behavior, and client logic.

"Every unexpected exception should be logged at every layer." Duplicate stack traces can overwhelm the service and obscure ownership.

"A handled exception appears automatically in metrics." Verify framework instrumentation and record handled failures deliberately.

"The request failed, so the write did not happen." Timeouts and response failures can occur after durable side effects; reconcile ambiguous outcomes.

"All 5xx errors are application bugs." Gateways, dependencies, capacity, and infrastructure can author or cause them.

A strong interview answer

I first identify the exact status and the layer that authored it, then scope by normalized route, version, instance, region, tenant class, payload shape, and start time. I preserve a representative trace, full causal chain, safe structured context, and shared-resource state before restart. I classify the failure as deterministic data/state, compatibility, dependency, resource exhaustion, concurrency, or configuration, and correlate it with deployment and dependency timelines. Spring exception handling produces a stable RFC 9457-style contract without exposing internals, while unexpected failures are recorded once with a correlation ID and bounded metrics. I control retries and exception logging during storms, reconcile ambiguous writes, and verify the fix through status, business outcome, metrics, traces, logs, and canary traffic.

Chapter summary

HTTP 500 diagnosis combines protocol ownership, exception causality, resource evidence, and business correctness. A safe error contract protects clients, but reliable operations require precise classification, bounded observability, and reconciliation of work whose outcome became ambiguous.

Revision Notes

- Identify which layer authored the 5xx response before debugging application code.
- Scope by route, version, instance, region, tenant class, payload, and start time.
- Measure both error ratio and absolute rate.
- Preserve a representative trace, causal chain, resource state, and change timeline.
- Use safe RFC 9457-style problem responses with stable types and correlation IDs.
- A broad exception handler is a containment boundary, not classification.
- Follow wrapper exceptions to the actionable cause and preserve causal chains.
- Log unexpected failures once at the owning boundary.
- Keep raw URL, user ID, exception messages, and stack traces out of metric labels.
- Handled exceptions may require deliberate observation recording.
- Mixed-version rollouts require backward-compatible schema and API changes.
- Treat historical data as a compatibility signal, not automatically invalid input.
- Distinguish database connection wait, execution, conflict, deadlock, and schema failure.
- Retry only transient, idempotent operations within a shared deadline budget.
- Shared resource exhaustion can produce many unrelated exception types.
- Serialization can fail after controller return or after response commitment.
- Async failures need explicit context, recording, cancellation, and response ownership.
- Rate-limit repeated stack traces while preserving total counts and representative samples.
- Reconcile writes that may have committed before the client observed an error.
- Verify the full error contract, observability, business result, and canary behavior.